

LISP

PLF

2023

ANDREI

COSMIN



INTRODUCERE

#11

comentariu

; comentariu

11 #

(format t "Hello world ~%.")
sau print

↳ me

LISP is
not case-sensitive

(defvar *name* (read)) → citește o var. cu numele name
de la tastatură

(defun hello-you (*name*)
 (format t "Hello ~a! ~%. " *name*))
)

am def. o functie cu numele
hello-you ce afișează
Hello și numele de la
tastatură

(+ 3 2 7 9) ; 3+2+7+9 și return. rezultatul

(function ID param1 param2 ...)

(+ 3 (-2))
one param.

#e (3.2 2) → complex number

2/3 → $\frac{2}{3}$ nu e 2:3

#\g → caracter

#\newline , #\space

"String"

Bool

- t → true
- nil → false

(/ 3 5) → 3/5

(/ 3.0 5) → 0.6

(1+ 3) → 4

Funcții

- string - uppercase
- sgnt
- reverse
- length

(subneg "four score and seven" 9)

(= 4 3) $\Rightarrow$ N12

(number "hello") ; is a number

Recursivitate

(+ 33 ((* 2.3 4) 9) $\Leftrightarrow$ 33 + (2.3 * 4) + 9

(+ (* (sin 0.3)
(sin 0.3))
(* (cos 0.3)
(cos 0.3))) $\rightarrow \sin(0.3)^2 + \cos(0.3)^2$

Structură control IF

(if (<= 3 2) (* 3 9) (+ 4 2 3))

$\Leftrightarrow$ if (3 <= 2) {
 return 3*9;
}
else {
 return 2+3+4;
}

↳ dacă nu am
param asta, se
va returna N12

(prog1 expr1 expr2 expr3 ...)

Loop in LISP

(dotimes (x 4) expr1
 de nou expr2
 --)

($\Rightarrow$) for (x=0; x<=4; x++)

(setf bag 2) $\rightarrow$ am creat o variabilă
 $\hookrightarrow$ variabilă globală

instr. let

(setf x 4) $\rightarrow$ x var. globală

(let ((x 3)) $\rightarrow$ x var. locală

(print x) $\rightarrow$ 3

(setf x 9)

(print x) $\rightarrow$ 9

(+ x x) $\rightarrow$ 18

)

} dacă apelez x
în afara let, se
va afișa 4

Funcții

(defun funct - name

(param1 param2 ...)

expr1

expr2

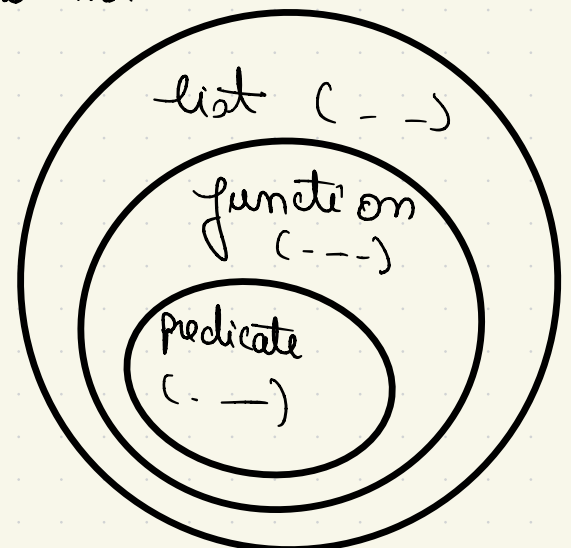
expr3 $\rightarrow$ asta se va returna

)

Structuri de date $\rightarrow$ only list

(quote (hello world 30))

'(hello amn 3)



(setf myList '((A B) C (D)))

myList = ((A B) C (D))

(car (car myList)) $\Rightarrow$ A
first first

(cdr (car myList)) $\Rightarrow$ (B)
rest first

(car (cdr myList)) $\Rightarrow$ C (C (D))
first rest

Reverse list

(defun my-reverse (li)

 (let (l2)

 (dolist (x li)

 (setf l2 (cons x l2))

)

 l2

)